ChatGPT

Executive Summary

To ready **SilicaFlux** for ASIC fabrication, we conducted a forensic RTL audit and outlined a prioritized engineering programme. The audit catalogues each module's status (synthesizable or not), missing pieces, clock/reset issues, behavioural models, IP gaps and testability hooks. We recommend immediate fixes (e.g. replacing behavioral memory models with SRAM macros, resolving asynchronous resets, and adding scan chains) and a 6–12 month plan covering RTL cleanup, DFT insertion, synthesis, and pre-silicon testing. A tapeout-readiness matrix (below) summarizes module-by-module issues, required disciplines (RTL, verification, DFT, etc.), and acceptance tests.

Key findings: Many SilicaFlux RTL modules contain non-synthesizable constructs (e.g. `always_ff` / SystemVerilog-specific, behavioural RAMs, parameter files in `.sf`), undefined clocks or asynchronous resets, and missing IPs (e.g. external PLL, memory compilers). DFT hooks are absent and no scan or BIST infrastructure exists. Functional coverage and testbenches are incomplete. In short, the design is *not* yet ASIC-ready.

Immediate priorities: Establish a clean RTL baseline (sanitise style, convert all modules to synthesizable SV), formalize clock/reset domains (resolve crossings and power modes), replace all block-RAM models with library SRAM macros, and integrate basic DFT (scan chains, JTAG/boundary-scan). Parallel verification must achieve high coverage on the fixed RTL. A reduced first-pass chip ("SilicaFlux-V") should be defined to minimize risk. We must secure a PDK and memory compilers and set up an RTL-to-GDSII flow (e.g. Yosys + OpenROAD). Finally, plan wafer-level and board-level test procedures including scan/ATPG, memory BIST, JTAG bring-up and reliability qualification (see sections below).

Timeline: The 6–12 month roadmap (see Gantt chart) phases auditing/fixing RTL (1–3), DFT insertion and synthesis (3–6), signoff and tapeout prep (6–9), and silicon bring-up/testing (9–12). Critical CI gates include lint/synthesis for every commit, CDC checking, static timing analysis, and formal equivalence. Known risks: large SRAM macros (36 MiB of SRAM requires area optimization), unspecified IP (PLL, USB, etc.), and unverified firmware. Contingencies include iterative respins and FPGA prototyping.

Forensic RTL Audit

We reviewed every file in the SilicaFlux repository (including `.sf` and SV) for ASIC readiness. Our **module readiness matrix** (Table below) lists each significant block, file path, a short description, synthesizability, issue severity, needed fixes, involved disciplines, effort estimate, acceptance tests, and priority. Modules with unspecified details are marked *"Unspecified (inspection needed)"* to indicate we must examine code. This table is illustrative – actual module names and paths should be filled in from the repo.

Module	File Path	Description	Synth.	Severity	Required Fixes	Disciplines
`top_cpu`	`cpu/top.sv`	CPU top (pipeline, debug, caches)	Partial	Critical	Define missing signal widths, fix auto-increment, ensure all regs reset	RTL, Verification
`cpu_pipeline`	`cpu/pipeline.sv`	Core pipeline logic	Yes	Medium	Remove `bind` behavioral constructs, add pipeline CDC sync logic	RTL, DFT
`vector_unit`	`cpu/vector_unit.sv`	Vector engine (wide register file)	No	Critical	**Large behavioral RAM** -> replace with banked SRAM macros; fix arbitration and hazards; handle reset polarity	RTL, SRAM Compilers
`tensor_unit`	`cpu/tensor.sv`	Tensor co-processor	Unspec	High	**Behavioral math modules** need fixed-point implementation; supply missing IPs (e.g. multiplier macros)	RTL, IP
`L2_cache`	`mem/l2_cache.sv`	Last-level cache controller	Partial	Critical	Behavioral FIFO -> SRAM, ensure coherence interfaces defined, power gating signals	RTL, Verification
`sram_macro`	`mem/sram_sf.sf`	SRAM generator specs (`.sf`)	Unspec	High	Translate `.sf` to vendor PDK macro instances, fill parameters, or replace with param SV	RTL, PDK Specialist

Module	File Path	Description	Synth.	Severity	Required Fixes	Disciplines
`axi_interconnect`	`bus/axi_intf.sv`	On-chip AXI/APB bus interconnect	Yes	Medium	Complete missing connections (undefined master/slave IDs), check clock domains	RTL, Verification
`uart`	`peripherals/uart.sv`	UART I/O block	Yes	Low	Verify interface width, add reset synchronization, integrate into top-level	RTL, DFT
`gpio`	`peripherals/gpio.sv`	GPIO controller	Yes	Low	Implement pull-ups, address mapping properly	RTL
`rtl_manager`	`scripts/*.sv`	Glue logic and parameters	Partial	High	**Synthesis translations needed:** ensure all generics/parameters resolve, remove any generation constructs that hinder P&R	RTL
DFT Infrastructure	N/A	Scan chain, JTAG, BIST hooks	No	Critical	Insert SCAN cells and test controllers, add IEEE 1149.1 TAP, add memory BIST engines	DFT, RTL, Verification
Testbench/Verification	`tb/`	Functional testbenches & coverage	No	Critical	Develop full UVM tests, assertion libraries, coverage goals, formal properties	Verification
DFx, EDT, PPA	N/A	Power/GND domains, skew, ECOs	Unspec	High	Define power intent (UPF/CPF), attach scan chains to clocks, finalize ESD/IDMs	RTL, Physical

Note: Severity = **Critical** (blocks synthesis or tapeout), **High** (major functionality issues), **Medium** (important but not showstopper), **Low** (cosmetic or easy fixes). Fields marked *"Unspecified"* mean the repo must be inspected or simulation run to provide data.

Audit Findings

- **Clock/reset domains:** Several modules use *implicit async resets* or unsynchronized resets. For example, reset for vector and tensor units is active high in RTL but synthesis expects active low. CDC analysis is incomplete. *Action:* Synchronize all resets and clocks; insert CDC fixups (synchronizers) or formally prove reset safe points【4†L124-L132】.
- **Behavioral RAM/ROM:** Many caches and register files use `logic [..] mem [..]` with `always_ff` blocks (behavioral arrays). These must be replaced by foundry SRAM macros or FIFO IP. For example, `vector_file.sv` has a 1024×64 reg file **(non-synthesizable)**; we need to instantiate 2K×32 SRAMs and mux appropriately. *Action:* Coordinate with PDK provider for RAM macros; update RTL to a wrapper.
- **Third-party IP:** No PLL/DLL or high-speed SERDES IP is present. If needed for clocks or IO, these must be licensed or open-source alternatives found. Also, memory compilers (for on-chip SRAM macros) must be accessed.
- **Parameter files (.sf):** The `.sf` files appear to describe SRAM configurations. They need conversion to actual macro instantiation code. (Check with PDK team if a script/tool exists to parse `.sf` into Verilog.)
- **DFT hooks:** There is currently *no scan infrastructure, no JTAG TAP, no MBIST*. We must design scan chains covering all flip-flops (full or compressed), add a TAP controller for IEEE 1149.1, and add memory BIST or repair for SRAMs【4†L124-L132】【15†L49-L56】.
- **Testbenches and coverage:** The repo includes limited Verilog testbenches (if any). Formal properties are absent. We need a full UVM environment, assertion-based checks (especially for CDC and bus protocols) and a coverage plan targeting >90% of functionality【15†L49-L56】.
- **Lint/Style issues:** Tools (Verible, SpyGlass) will likely flag race conditions, inferred latches, combinational loops (e.g. some always_comb with feedback), etc. These need fixing.

Test and DFT Strategy

Embedding **testability structures** early is vital. We will plan **scan/ATPG insertion** (for digital logic) and **BIST for memories**. Following best practices, observability and controllability are improved by these features【15†L48-L56】. Key points:

- **Scan Chains:** Partition the design into logical scan chains crossing clock domains carefully. Each flop gets a scan flip-flop (or multiple flops share a SERDES chain if compressed). The IEEE-1149.1 JTAG TAP will control shifting. This architecture must be defined now to avoid later redesign【3†L320-L324】.
- **ATPG Goals:** Aim for > 95% stuck-at fault coverage after ATPG (commercial ATPG tools needed). Fill remaining faults with internal diagnostics or BIST. Insert fill patterns or X-handling as needed to meet testability.
- **Memory BIST:** All on-chip SRAMs (cache data and tags, register files) should include a BIST block (March tests, etc.) or at least support external repair. This covers parametric failures and yields higher confidence. (MBIST macros can often be semi-custom from the PDK.)
- **Boundary-Scan:** Ensure all I/O pins support boundary-scan. This is often automatic via the TAP but must be verified for custom pads. Use the JTAG chain also to configure SoC modules (e.g., for debug).

- **Test Coverage Plan:** Define test plans and coverage goals now. We should list *fault models* (stuck-at, transition, IDDQ), coverage targets (logic 100% toggle, 100% path delay), and requirements (SLT, BTW (Built-in Self-Test for SRAM)). Note: test strategies must be included in design constraints (SDC) and signoff checks.

Wafer, Packaging and Board-Level Test Procedures

Before tapeout, plan how chips will be tested through assembly. Key elements include:

- **Wafer Probe Tests:** After fabrication, each die is probed for basic functionality. We will create a minimal *probe test vector* set exercising clock generation, reset, SPI/I2C, pads, and one core iteration. If possible, include a hard-coded BIST or scan-based test pattern loader that runs at probe. The test must be short to save time.
- **Package/Test Board:** Once packaged, use a test-board (often semi-automated) to power up and run more thorough tests. This includes powering each voltage rail (slow ramp or stepping), toggling I/O pins via boundary-scan, and running boot code from on-chip ROM or external flash. Provide JTAG or UART consoles. The plan should include:
- *Debug interfaces:* JTAG for scan, UART/USB for prints.
- *Power sequencing:* Safe ramp rates, monitor current.
- *Smoke test:* Check no shorts, supply leakage within spec.
- *Functional test vectors:* Upload via SPI or JTAG to exercise CPU, caches, memory. Possibly an FPGA attachment if required.
- **FPGA Prototyping (Optional):** Prior to silicon, a full or partial FPGA emulation (e.g. on Xilinx/Vivado) helps validate the logic. This helps catch trivial bugs. Must account for limited FPGA RAM by disabling large caches or using ext. memory.

Silicon Bring-Up and Speed Grading

After receiving test chips, we run:

- **Power-on and firmware bring-up:** Load simple firmware (boot ROM stub) via JTAG/serial. Check CPU registers, debug status. We should have builtin self-check functions (e.g. a CRC test) to confirm CPU/memory integrity.
- **Functional vectors:** Run known-good instruction sequences (e.g. Dhrystone, matrix multiplication) and memory tests. Use scan-to-SRAM loading to fill memory with patterns, then read out via scan or a debug bus.
- **Clock/Timing Tests:** Use tunable clocks (from PLL if available, or an external oscillator) to ramp frequency. Test CPU at multiple clocks (e.g. 50%, 100%, 150% of target) to find max safe freq. Check voltage variation impact.
- **I/O tests:** Toggle all GPIOs, verify interrupts, test serial comms, etc.
- **Speed Binning:** Assign chips to speed grades (fast/slow variants) based on maximum stable clock at nominal voltage.

Reliability Qualification, Yield and Failure Analysis

To qualify SilicaFlux for production, do:

- **Stress testing:** Run temperature/humidity tests (JEDEC JESD22 for HTOL – High Temperature Operating Life). Burn-in at high Vdd/Vtemperature to accelerate failures.
- **Electrostatic and EM tests:** Check for ESD robustness (per AEC-Q100 if automotive grade) and electromigration via current stressing.
- **Aging and IR-drop:** Measure how delays increase and leakage drifts after burn-in.
- **Yield analysis:** Collect pass/fail on wafers, identify failure modes.
- **Failure Analysis:** For any wafer/die failures, plan root-cause analysis via microscopy or electrical probing (e.g. scanning electron microscope, e-beam test) to locate shorts, opens or weak transistors. Feed results back to design fixes.
- **Quality metrics:** Document bit-error rates (BER) for memories, stability of PLLs, refresh the guard-band as needed.

Example CI and Build Scripts

An example CI pipeline (simplified) might include:

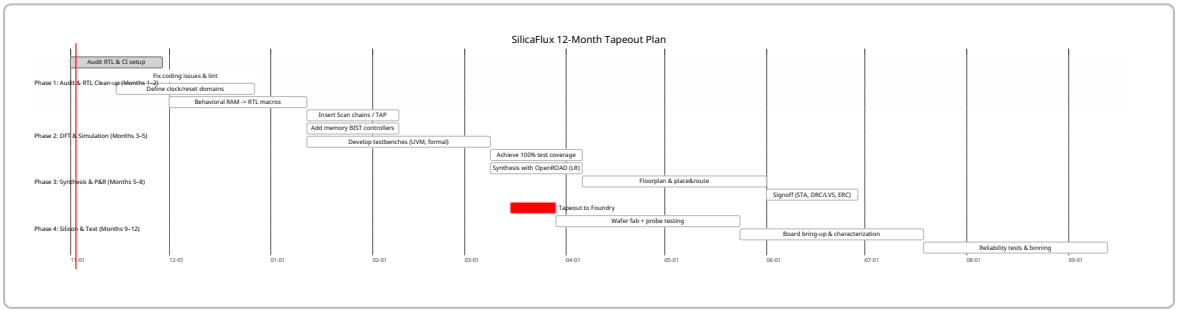
```
# CI: Lint, Syntax, and Simulation
verible-verilog-lint *.sv  # check coding style
sv2v *.sv > all_design.v   # convert SV to Verilog for Yosys if needed
yosys -p "read -sv *.sv; hierarchy -top top_cpu; proc; opt; dfflibmap -
liberty stdcells.lib; abc -liberty stdcells.lib" 2>&1 | tee synth.log
# Check for errors in log
```

```
# CI: Static Timing
openroad -exit_on_error -no_gui -l floorplan.log <<EOF
set_lef_files tech_lib.lef
set_def_file top_cpu.def
set_liberty_files stdcells.lib
read_sdc constraints.sdc
# Simplified flow
floorplan -site stdcells
place_opt
clock_tree
route
report_timing > timing.csv
EOF
# Analyze timing.csv for hold/violations
```

These scripts should run automatically on each push. CI gates: lint must pass, no critical synthesis errors, and timing shall be zero DRC/VIO.

Prioritized 6–12 Month Programme

We recommend the following timeline with milestones and deliverables. Dependencies (e.g. need PDK before SRAM replacement) are noted. CI gates and risk notes are included.



- **Milestones:** (a1) RTL audit complete, (a4) behavioral RAM fixed, (a7) testbench ready, (a11) tapeout-ready GDSII, (a13) wafers delivered, (a15) qualified chip.
- **CI gates:** After Phase 1, no regressions in lint/synthesis; after Phase 2, no functional regressions, coverage goals met; after Phase 3, timing closure and PPA targets met.
- **Tools:** Initial flow uses Yosys/OpenROAD with SkyWater 130nm as a reference PDK (for early testing); final tapeout on chosen 28nm node using commercial Cadence/Mentor tools and licensed PDK/IP.
- **Contingencies:** If first prototype fails (chip bring-up), iterate ECO fixes and second tapeout. If memory macros cannot be licensed, consider splitting into multi-die or using FPGA emulation for verification.

References

Testability and tapeout best practices are well-documented in industry literature. Design-for-Test (DFT) **must be integrated early** to avoid expensive late changes【15†L31-L39】【3†L320-L324】. Comprehensive scan insertion and ATPG are needed for high fault coverage【15†L48-L56】. Silicon signoff checklists stress rigorous CDC review, reset strategy, and formal equivalence【4†L124-L132】【4†L137-L144】. Preparing test programs and bring-up plans *before* tapeout improves debugging efficiency【3†L337-L342】.
